AscendGYM: A Cost-Aware Kernel Optimization Benchmark for Real-World Workloads on Ascend NPUs

AI Training Platform Team, Shenzhen Loop Area Institute

Abstract

Kernel optimization is essential for efficient large language model (LLM) training and inference on Ascend NPUs. Existing Ascend kernel benchmarks primarily focus on kernel generation, with less emphasis on the ability of agents to optimize existing implementations under input configurations drawn from real workloads. To address this requirement, we introduce AscendGYM, a benchmark whose initial suite comprises 48 kernels across three difficulty levels. Each kernel is paired with test cases whose data types and tensor shapes are collected from real-world training and inference tasks. Consequently, performance improvements on AscendGYM map directly to the corresponding workloads. In terms of usability, two evaluation modes are supported in AscendGYM: model evaluation and model–harness evaluation. The kernel optimization capabilities of both trained models and custom agents are allowed to be measured. It provides a complete execution environment that closely reproduces how Ascend kernels are invoked in practical pipelines. For model evaluation, it additionally includes a built-in harness equipped with search orchestration and experience accumulation. Finally, we propose a new metric, the Speedup–Token Score (STS), which quantifies performance gains relative to token cost. By jointly considering speedup and cost, STS enables the assessment of practical kernel optimization capability of a model or harness under an acceptable budget. AscendGYM is available at https://github.com/SLAI-AITP/AscendGYM.

1 Introduction

Efficient large language model (LLM) training and inference depend on the performance of the underlying kernels. On Ascend NPUs, improving kernel implementations for real workloads is an important engineering step toward greater system efficiency (Li et al., 2026). Such optimization typically starts from existing code and requires developers to analyze performance bottlenecks in the context of target input configurations and hardware characteristics, then repeatedly modify and execute implementations to validate improvements (Andrews & Witteveen, 2025). The process therefore relies on specialized expertise and sustained engineering effort (Tschand et al., 2025).

Coding models and agents offer an opportunity to automate this process (Baronio et al., 2026). Numerous benchmarks have been proposed to evaluate their capabilities. KernelBench evaluates the ability of models to generate efficient kernels for given PyTorch workloads (Ouyang et al., 2025), while NPUKernelBench and CANN Bench extend kernel-generation evaluation to Ascend, assessing compilation, correctness, and performance (Cao et al., 2026; Gao et al., 2026). We focus on a concrete optimization evaluation task: given a correct kernel implementation, can the specific agent further improve its performance for a target workload within an acceptable budget while preserving correctness? To study this task, we introduce **AscendGYM**, a kernel optimization benchmark for Ascend NPUs, whose design centers on the following questions:

How should evaluation tasks be designed? Optimization priorities for the same kernel can vary with input configurations (Ye et al., 2025). For example, attention computations process

For any questions or feedback regarding this technical report, please contact Dongfang Li (lidongfang@hit.edu.cn) and Baotian Hu (baotianhu@slai.edu.cn).

different data sizes during prefill and decoding, which can favor different ways of organizing computation (Kamath et al., 2025). FlashInfer-Bench draws workloads from real inference traces, illustrating the connection between input configurations and application scenarios (Xing et al., 2026). AscendGYM selects kernels and their input shapes, data types, and computation parameters from real training and inference workloads, and provides existing implementations together with corresponding test cases. Each task thus has a concrete optimization target and defined validation conditions, with performance gains measured relative to the initial implementation's latency under the same conditions.

How can evaluation be made reliable? Whether modified code compiles, produces correct results, and runs faster must be checked separately through actual execution (Li et al., 2025b). Prior work shows that compiler and profiling feedback can help models correct errors and improve implementations (Ouyang et al., 2025). On Ascend, this process also involves coordinated builds of host and device code, followed by execution and measurement on NPUs (Cao et al., 2026). We design **AscendForge**, AscendGYM's built-in execution sandbox, to integrate candidate compilation, correctness checking, and performance measurement into a programmatically accessible evaluation service. It returns execution feedback under predefined validation conditions and measurement protocols, and records optimization trajectories to support iterative agent optimization and result verification.

How should optimization performance be assessed? Repeated sampling and iterative refinement can improve results for some models, while requiring additional inference and execution resources (Ouyang et al., 2025). Two systems may achieve the same final speedup yet differ substantially in the search effort required: one may identify improvements within a few attempts, whereas the other may require an extended search (Cha et al., 2025). Final speedup alone does not capture this distinction. **Speedup–Token Score (STS)**, which combines performance gains within a given budget with cumulative token consumption at each improvement, is designed to assess optimization under resource constraints. The metrics for performance, resource consumption, and correctness are also reported separately to distinguish the gains achieved, the resources invested, and whether candidate implementations pass validation.

AscendGYM currently comprises **48 kernel optimization tasks** across **three difficulty levels** and supports **two evaluation modes**: comparing models using a default harness, and evaluating model–harness combinations as complete systems. Multiple leading models paired with the commonly used harnesses are systematically evaluated on the **17 tasks in Batch 1**. Approximately 70% of the evaluated combinations achieve geometric mean speedups below 1.2×, indicating that substantial improvements across a broad range of existing implementations remain challenging. Gain trajectories under different budgets, together with differences in performance, cost, and correctness, also show that final speedup alone does not fully characterize optimization behavior. These findings provide empirical support for the benchmark's task and metric design.

The main contributions of AscendGYM are:

1. **AscendGYM** is a benchmark for Ascend NPUs that uses kernels and input configurations from real workloads to evaluate the ability of agents to optimize existing implementations.
2. **STS** is the key metric of AscendGYM, combining performance gains with the token consumption required to obtain them to assess optimization under a given budget.
3. **AscendForge** built in AscendGYM supports iterative agent optimization through unified interfaces for compilation, correctness checking, performance measurement, and feedback.

2 Background and Related Work

Existing kernel generation benchmarks primarily evaluate whether models can construct correct and efficient implementations from computational requirements. KernelBench (Ouyang et al., 2025) specifies tasks using PyTorch programs and asks models to generate semantically equivalent GPU kernels. TritonBench (Li et al., 2025a) focuses on Triton kernel generation, while MultiKernelBench (Wen et al., 2025) extends evaluation to multiple hardware platforms. For Ascend, NPUKernelBench (Cao et al., 2026) provides functional descriptions, input and output requirements, and code templates to be completed, and evaluates the correctness and performance of the resulting implementations. CANN

Bench (Gao et al., 2026) provides operator specifications, test cases, reference implementations for correctness checking, and performance baselines, and assesses candidate kernels against hardware performance limits. These benchmarks establish the computational requirements, correctness criteria, and performance targets that generated implementations need to satisfy.

Some evaluations start directly from existing kernel implementations and assess agents' ability to improve their performance. For example, AMD evaluates optimization of existing Triton kernels on TritonBench-modified and ROCm-bench, requiring agents to improve execution performance on AMD GPUs while preserving correctness (Wang et al., 2025). These settings use existing kernel source code as the optimization target and measure performance improvements through speedup relative to the initial implementation.

PRAGMA (Lei et al., 2025) incorporates execution feedback and hardware profiling into an iterative reasoning loop. AutoKernel (Jaber & Jaber, 2026) profiles models to prioritize bottleneck kernels, then validates and benchmarks successive Triton or CUDA C++ implementations.

Beyond the starting implementation and target platform, the relationship between kernels, input configurations, and actual applications also affects the practical relevance of performance evaluation. SOL-ExecBench (Lin et al., 2026) extracts kernel tasks from real models and uses hardware performance limits to assess the remaining optimization headroom. Atrex-Bench (Yang et al., 2026) and FlashInfer-Bench (Xing et al., 2026) construct evaluation tasks from execution traces of real inference workloads, connecting kernels and their input configurations to specific deployment scenarios. These efforts illustrate how workload-driven task design gives kernel performance evaluation a concrete application context.

AscendGYM focuses on evaluating the optimization of existing kernel implementations on Ascend NPUs. Its tasks comprise kernels used in real large-model workloads, with input shapes extracted from the corresponding workloads to reflect concrete execution requirements. Each task provides complete, runnable initial source code and requires agents to improve performance while preserving correctness, using the measured latency of the initial implementation under the same input configuration as the reference. AscendGYM includes the AscendForge execution environment, which provides shared support for candidate compilation, correctness checking, and performance measurement. Under explicit evaluation protocols and budgets, we compare the optimization performance of different models and harnesses, considering performance gains, correctness, and resource consumption.

3 AscendGYM

AscendGYM is a kernel optimization benchmark for Ascend NPUs. It evaluates the ability of model–harness combinations to improve existing kernel implementations. Its four components are the task set and test cases, the AscendForge evaluation environment and tools, evaluation metrics, and a shared evaluation workflow. Together, they provide tasks and execution feedback and assess optimization outcomes under consistent rules.

3.1 Task Selection and Test Case Design

AscendGYM organizes kernel optimization tasks into three batches of increasing difficulty, progressing from largely independent kernels to multistage fusion and more complex execution coordination. Batch 1 focuses on memory access optimization for basic kernels. Batch 2 covers multistep computations, such as normalization and quantization. Batch 3 targets complex attention mechanisms and fused computations, including sparse attention. This report evaluates the 17 kernel optimization tasks in Batch 1; results for Batches 2 (17 kernels) and 3 (14 kernels) will be reported in future updates.

Task selection prioritizes bottleneck kernels with substantial performance impact in real workloads. Each test case preserves the connection between kernel inputs and their deployment context, including the target model, execution phase, hardware and parallelism configurations, and the resulting tensor shapes, data types, and computational parameters. This context links measured kernel speedups to the corresponding operator invocations in real workloads, aligning evaluation with deployment requirements.

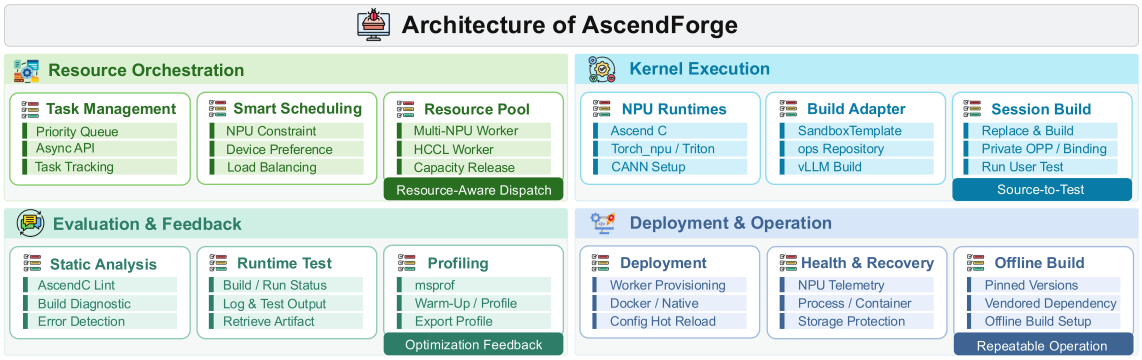


Figure 1: Architecture of AscendForge, integrating resource orchestration, kernel execution, evaluation and feedback, and deployment management for Ascend kernel optimization.

For example, the `AddRMSNorm` task captures residual addition followed by RMS normalization during a forward pass of Qwen3-0.6B (Yang et al., 2025) inference. The test case uses the prefill phase with no quantization, eager execution, a single NPU, TP 1, BF16, a batch size of 1, and an input sequence length of 32 tokens. The branch output and residual tensors both have shape $[32, 1024]$, and the normalization weight has shape $[1024]$. Profiling of its workload shows 56 invocations per forward pass, accounting for 8.77% of total computation time and ranking among the top-5 kernels by cumulative execution time. Preserving this execution context supports assessment of contribution to workload-level performance.

AscendGYM evaluates performance across multiple deployment cases, the best achievable performance for specific shapes and scenarios, and the optimization cost required to obtain these gains. Together, these dimensions characterize cross-case applicability, shape specialization, and optimization efficiency.

3.2 Evaluation Environment and Tools

AscendGYM includes AscendForge, an Ascend-native execution sandbox for candidate compilation, correctness checking, and performance measurement.

Like other code evaluation benchmarks, AscendGYM must verify that implementations preserve the required computational semantics after successful compilation and execution. KernelBench (Ouyang et al., 2025) demonstrates the importance of executing candidate implementations and returning compiler feedback and correctness results to the model. SOL-ExecBench (Lin et al., 2026) incorporates isolated execution and checks against inadvertent modifications to the measurement procedure into its evaluation harness. FullStack Bench (Cheng et al., 2024) released SandboxFusion as reusable infrastructure for executing and evaluating generated programs across programming languages. Building on SandboxFusion (Cheng et al., 2024), AscendForge introduces adapters that manage dependencies among host-side registration, device-side kernels, the target system-on-chip (SoC), and custom kernel packages, enabling compilation and Python interface binding for NPU devices.

As shown in Figure 1, AscendForge comprises four layers for kernel building and execution, resource scheduling, development feedback, and deployment and operations.

Execution and Build Layer. This layer supports Ascend C and Python NPU execution backends, with adapters for CANN kernel repositories (CANN_ops_math, n.d.; CANN_ops_cv, n.d.; CANN_ops_nn, n.d.; CANN_ops_transformer, n.d.), an Ascend C custom kernel build template, and projects such as vLLM Ascend (vLLM Ascend Contributors, n.d.). It automates source submission, compilation and installation, Python interface binding, and test execution.

Resource Orchestration Layer. This layer manages evaluation requests through a scheduling proxy, task queues, and multiple execution nodes. It allocates resources according to the required NPU count, device model, and node preferences, and supports concurrent evaluation through resource reservations and load management.

Development Feedback Layer. This layer integrates static code analysis and performance profiling, and returns build logs, correctness results, performance measurements, and test artifacts. These outputs help agents diagnose errors, identify performance bottlenecks, and guide the next optimization iteration.

Deployment and Operations Layer. This layer provides hardware health monitoring, configuration hot reloading, and automatic process recovery, together with offline dependency preparation and version verification, to support service availability and environment consistency.

The interface design of AscendForge preserves the build conventions of the underlying projects and provides a session-specific workspace for each build. AscendForge separates the client, scheduling proxy, and execution workers. The client packages the source code and test driver, and the scheduling proxy selects a compatible worker using device count, SoC type, and worker group as scheduling constraints. Node and device preferences are also considered when this information is available. The worker builds and executes the kernel; the test driver first verifies correctness and then collects performance metrics. Compilation and execution results are reported separately during testing, allowing the model to distinguish build failures, numerical accuracy failures, and performance regressions. The harness uses this feedback to guide the next optimization iteration, while AscendGYM reevaluates the candidate implementation at the end of the current run.

3.3 Metric Design

While conventional kernel benchmarks primarily evaluate final performance, AscendGYM evaluates both optimization outcomes and the resources required to achieve them. Its evaluation framework includes joint performance–budget metrics, standalone performance and budget metrics, and a reliability metric. In particular, the joint metrics capture optimization quality and efficiency within a single score.

3.3.1 Joint Performance–Budget Metrics

Speedup–Token Score (STS). AscendGYM evaluates the kernel optimization capabilities of LLMs and agent harnesses under explicit resource budgets, reflecting the practical constraints of kernel optimization. STS jointly measures the best speedup achieved within a token budget and the token consumption required to obtain improvements:

$$\mathrm{STS} = \underbrace{100 \log_2 S_n}_{\text{final performance gain}} - \underbrace{50 \sum_{k=1}^{n} \frac{B_k}{B_{\max}} \Delta g_k}_{\text{penalty for delayed improvements}} \tag{1}$$

Here, $B_{\max} > 0$ is the token budget, and n is the number of candidate evaluations completed within that budget. B_k denotes the cumulative token consumption when evaluation k completes, including tokens spent on failed attempts and attempts that yield no improvement. S_k denotes the best correctness-validated speedup available after evaluation k, relative to the reference implementation. The reference implementation is retained when no improvement is found, so $S_0 = 1$ and $S_k \geq 1$.

The incremental log-speedup gain is

$$\Delta g_k = \log_2 S_k - \log_2 S_{k-1}. \tag{2}$$

The first term in Eq. (1) rewards final performance, whereas the second penalizes improvements obtained later in the token budget. Thus, for the same final speedup, earlier improvements receive a higher score. Higher STS indicates better budget-aware optimization under the same evaluation conditions. If no improvement is achieved, $\mathrm{STS} = 0$.

Cost-to-Gain (CTG). CTG measures the additional token consumption required to increase speedup by a specified fraction relative to a reference checkpoint:

$$\mathrm{CTG}(\delta; r) = \min_{\substack{r \leq k \leq n \\ S_k \geq (1+\delta) S_r}} (B_k - B_r), \qquad \delta \in \{0.01, 0.05, 0.10\}. \tag{3}$$

Here, r is the reference checkpoint, and $B_k - B_r$ is the token consumption accumulated since that checkpoint. The thresholds correspond to relative speedup gains of 1%, 5%, and 10%. We define the minimum of an empty set as $+\infty$, indicating that the target gain was not reached within the budget. Lower CTG indicates greater token efficiency.

3.3.2 Performance Metrics

AscendGYM reports GM Speedup and the fast_p metric introduced by KernelBench (Ouyang et al., 2025). For N evaluation tasks, let S_{i,n_i} denote the best correctness-validated speedup achieved on task i within the fixed budget.

$\text{fast}_\mathbf{p}$. fast_p measures the fraction of tasks for which a correct implementation achieves a speedup strictly greater than p:

$$\text{fast}_p = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}\left[S_{i,n_i} > p\right], \qquad p \geq 1. \tag{4}$$

Here, $\mathbb{1}[\cdot]$ is the indicator function. Correctness is implicit in the definition of S_{i,n_i}. Tasks with no improvement retain a speedup of 1 and therefore do not contribute to fast_p for $p \geq 1$.

Geometric Mean Speedup (GM Speedup). GM Speedup summarizes performance across tasks using their geometric mean:

$$\text{GM Speedup} = \left(\prod_{i=1}^{N} S_{i,n_i} \right)^{1/N}. \tag{5}$$

Each task receives equal weight, and the reference implementation is retained when no improvement is found.

3.3.3 Budget Metrics

AscendGYM measures optimization cost using cumulative and mean token consumption, LLM interaction rounds, and wall-clock time. All budget metrics include failed attempts to capture the full resource and time costs of obtaining optimization results.

3.3.4 Reliability Metrics

Candidate Pass Rate. Candidate Pass Rate measures how reliably an LLM or agent harness produces correct candidates. It is defined as the fraction of evaluated candidates that successfully compile, execute, and pass all required correctness checks. Compilation failures, runtime errors, timeouts, and incorrect outputs count as failures.

Task Pass Rate. Task Pass Rate measures the fraction of attempted tasks with a valid baseline measurement and at least one candidate that passes the required correctness checks during search. Tasks with no candidate submission or no passing candidate remain in the denominator. This measures the coverage of correct outcomes across tasks, while Candidate Pass Rate measures correctness across individual candidate evaluations.

When independent repeats are available, they can additionally assess the reproducibility and stability of optimization outcomes.

3.4 Evaluation Workflow

3.4.1 Evaluation Modes

AscendGYM supports two evaluation modes—model–harness evaluation and model evaluation—under the same task configurations and validation environment.

Model–Harness Evaluation. Users configure an existing coding agent, such as Claude Code, Codex, or OpenCode, or integrate a custom harness. AscendGYM evaluates the resulting model–harness pair as a complete system.

Model Evaluation. Users provide an LLM API, and AscendGYM runs the model with AuraKernel, hereafter referred to as `Default`, as the default harness. AuraKernel supports strategy search and experience summarization. Holding the `Default` configuration fixed provides consistent evaluation conditions across models. The configuration of `Default` is detailed in Appendix A.

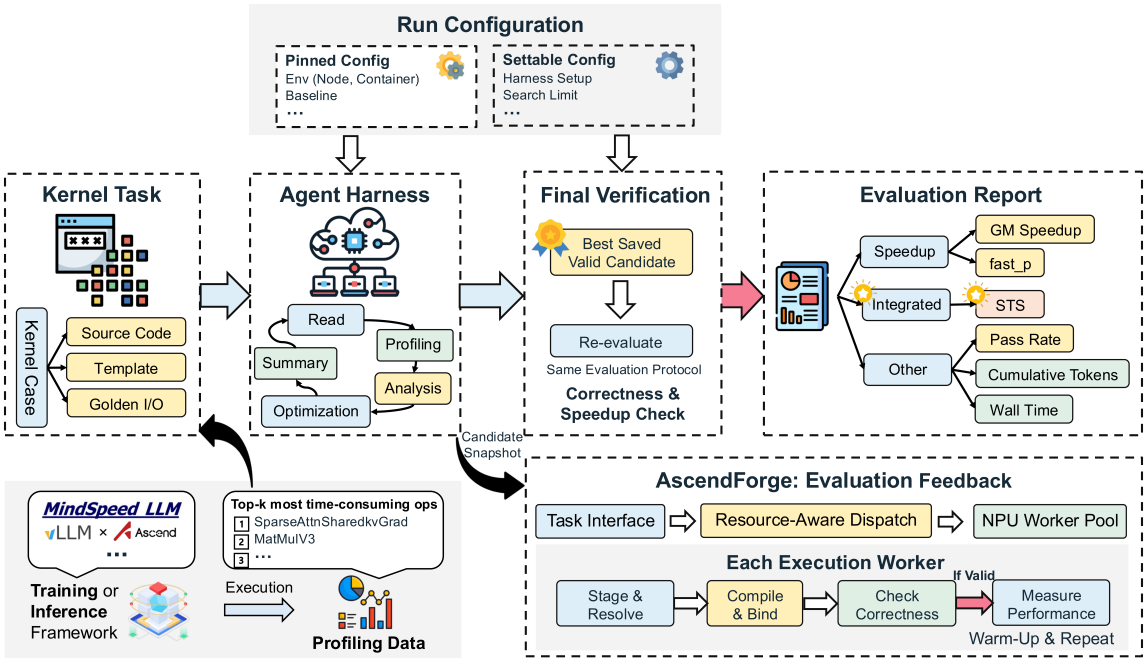


Figure 2: Overview of the AscendGYM workflow.

3.4.2 Evaluation Procedure

Figure 2 illustrates how AscendGYM constructs kernel optimization tasks from real training and inference workloads. We collect profiling data from workloads running in Ascend frameworks, such as MindSpeed-LLM and vLLM-Ascend, and identify the top-k bottleneck kernels. Each kernel optimization task includes the source code of the baseline, template metadata specifying input and output tensor shapes, test inputs, and reference outputs generated by the corresponding CPU implementation. These artifacts support numerical correctness checks and isolated kernel performance measurement.

To launch an evaluation, users configure the execution environment and optimization settings. AscendGYM first measures baseline performance, then enters an iterative loop of bottleneck analysis, code refinement, performance measurement, and experience summarization. Each optimized candidate is submitted to the AscendForge sandbox for compilation, Python interface binding, and correctness checks. Candidates that pass these checks are profiled, and the resulting measurements guide subsequent optimization.

Search terminates when the resource budget is exhausted or the maximum iteration count is reached. The system then revalidates the best implementation and uses the performance measurements and optimization traces to compute the metrics defined in Section 3.3.

4 Experiments

4.1 Setup

Evaluated Models and Harnesses. We conduct experiments using the two evaluation modes defined in Section 3.4. For model–harness evaluation, we evaluate selected combinations of models and three coding-agent harnesses: Claude Code (Anthropic, n.d.), Codex CLI (OpenAI, n.d.), and OpenCode (OpenCode Contributors, n.d.). The evaluated models are Claude Opus 5 (Anthropic, 2026), GPT-5.6 Terra (OpenAI, 2026b), GLM-5.3 (Z.ai, 2026), Qwen3.5-122B-A10B (Qwen Team, 2026a), Qwen3.6-35B-A3B (Qwen Team, 2026b), and DeepSeek-V4-Flash (DeepSeek-AI et al., 2026).

For model evaluation, we use the `Default` harness provided by AscendGYM to evaluate the kernel optimization capabilities of GPT-5.6 Luna (OpenAI, 2026a) under a fixed workflow and tool interface.

Detailed parameters and context management settings for `Default` are provided in Appendix A. The shared task prompt for model–harness evaluation is presented in Appendix B.

Tasks and Baselines. Task sets and test cases adopted in evaluation are described in Section 3.1, with the same initial implementations and test cases across configurations. For model mode, configuration-specific scoring and measurement protocols are detailed in Appendix A. The evaluated tasks are drawn from CANN `ops-nn` and `ops-transformer` (CANN_ops_nn, n.d.; CANN_ops_transformer, n.d.), and vLLM Ascend (vLLM Ascend Contributors, n.d.). Their task packages specify source versions, test inputs, and numerical comparison rules.

For each task, the baseline is the initial Ascend C implementation shipped with the task package. We measure its execution latency in the evaluation environment and use this measured latency as the performance reference for candidate implementations. All speedups are computed relative to the corresponding task's initial implementation.

Optimization Budgets and Final Re-evaluation. Each model–harness optimization run allows at most 80 candidate evaluations, with a soft target of exploring 50 substantively different optimization solutions specified in the prompt. After reaching this target, agents may continue optimizing within the remaining evaluation budget; they may also finish before reaching it if no reasonable optimization direction remains, and explain the reason. Failed submissions count toward the evaluation budget, whereas baseline measurements are excluded; no hard limits are imposed on total token consumption or wall-clock time.

For these runs, after the search ends, the final selected implementation undergoes five additional performance evaluations. These evaluations do not consume the search budget, provide no feedback for further optimization, and are not counted as independent optimization runs.

Correctness and Performance Measurement. Candidate implementations are validated using each task's predefined test scripts and numerical error criteria. Only implementations that pass all required checks are considered valid results.

Baseline and candidate implementations follow the same measurement protocol: each evaluation executes eight rounds, with 3 rounds for warm-up and the remaining rounds to compute the arithmetic mean of device-side kernel execution times. Timings are collected using `msprof`. Hardware and software details are provided in Appendix C.

4.2 Results and Discussion

Following the workflow in Section 3.4, we conduct model–harness evaluation and model evaluation to examine how the metric suite characterizes practical optimization behavior. We focus on how cost constraints affect the assessment of optimization outcomes, how aggregate performance relates to task coverage, and what resource-cost and correctness metrics reveal beyond performance alone. We also use the gains achieved by current systems to assess the challenge of further optimizing existing implementations.

4.2.1 Model–Harness Evaluation

Following Section 3.3, we analyze the results through four metric categories: joint performance–budget, performance, budget, and reliability metrics.

Joint Performance–Budget Metrics. Figure 3 presents three usage scenarios defined in this study: tight, moderate, and generous per-task budgets of 10M, 20M, and 50M tokens, respectively. Combinations exhibit different gain trajectories as the budget increases. For example, Opus 5 achieves similar STS with Claude Code and OpenCode at 10M tokens. With larger budgets, Claude Code's performance component continues to grow, whereas OpenCode obtains more limited additional gains. Reporting only performance at the end of the full search therefore does not adequately describe which gains are available under different budgets.

Cost deductions further reveal differences in the investment required for these gains. At 50M tokens, Claude Code/Opus 5 has a larger performance component than OpenCode/Opus 5 but also incurs a larger cost deduction. The deduction narrows the score gap without eliminating the former's

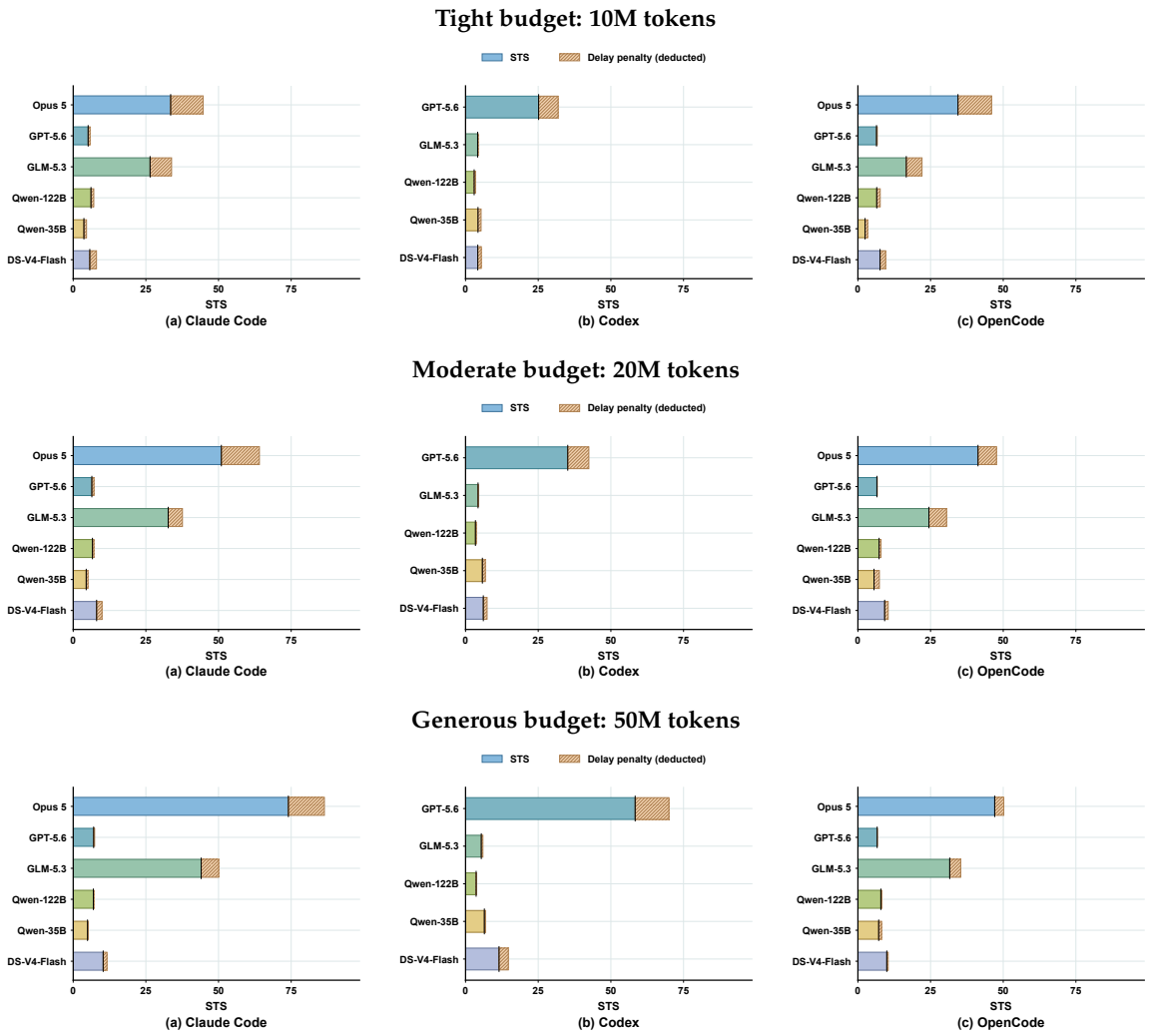


Figure 3: STS under three token budgets. Solid bars show scores; hatched segments show cost deductions.

advantage (Figure 3). Assessing optimization should account for both useful results obtained with limited expenditure and substantial gains enabled by additional investment. Scores and their components across the three budgets make this trade-off visible.

The CTG results further show that lower attainment cost and broader task coverage do not always coincide. At a 50M-token budget and a 10% gain target, Opus 5 with OpenCode and Claude Code has median costs of 3.8M and 6.6M tokens over attained tasks, respectively, but the former reaches the target on only a subset of tasks while the latter covers all tasks (Figure 4). Because the attained task sets differ, the lower conditional median does not establish greater efficiency on the same tasks. Reporting CTG together with attainment counts captures both the cost and coverage of target gains, avoiding choices based solely on costs among successful tasks.

Performance Metrics. GM speedup and fast$_p$ jointly reveal differences between aggregate gains and their distribution (Figures 5 and 6). Claude Code/Opus 5 and Codex/GPT-5.6 both accelerate all tasks, but seven and three tasks, respectively, exceed 2$\times$ speedup. Identical coverage of accelerated tasks therefore does not imply identical improvement magnitudes; increasing the threshold allows fast$_p$ to distinguish these outcomes. Aggregate speedup should consequently be interpreted alongside coverage at different thresholds to assess whether gains meet application requirements.

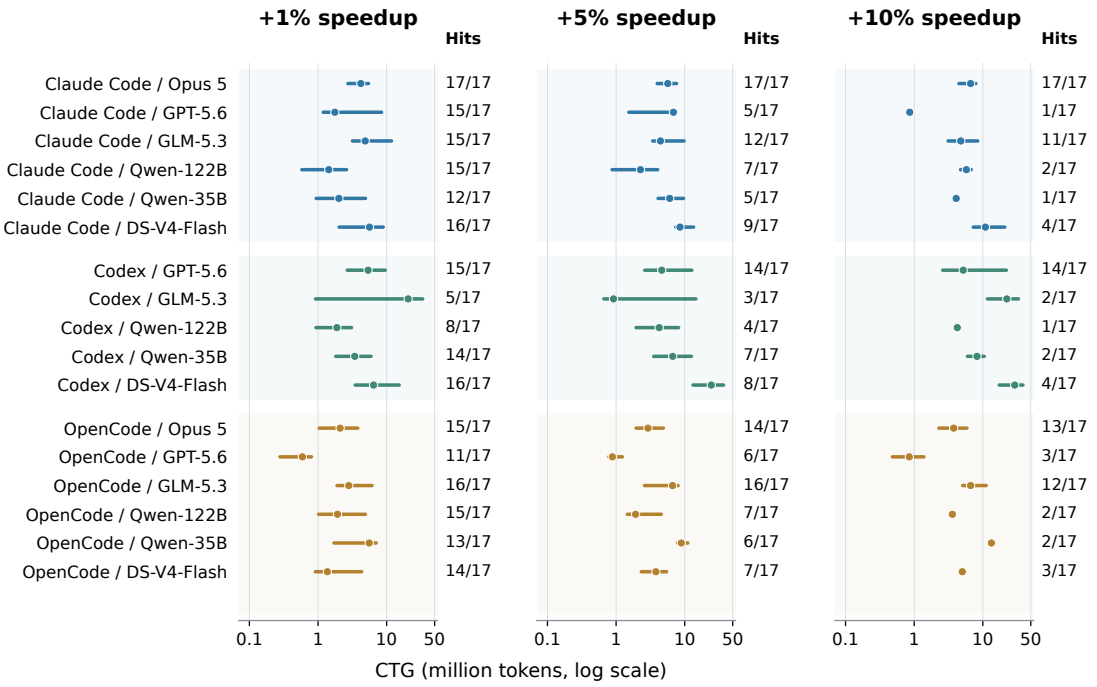


Figure 4: CTG at a 50M-token budget. Dots and lines show medians and interquartile ranges over attained tasks; counts give attainment out of 17 tasks in Batch 1.

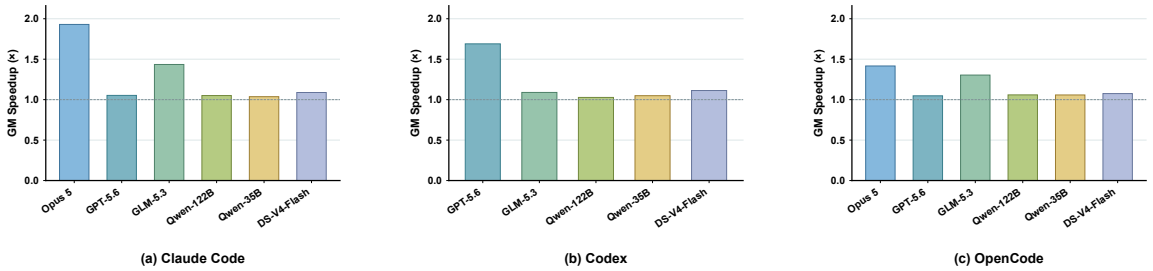


Figure 5: GM speedup over full search trajectories, where dashed lines mark 1×.

Overall, current systems still achieve limited gains on the benchmark. Approximately 70% of the evaluated model–harness combinations have GM speedups below 1.2×, with gains largely concentrated in modest improvements and limited task coverage at higher speedup thresholds (Figures 5 and 6). Under the given initial implementations and evaluation conditions, achieving substantial improvements across a broad range of tasks remains challenging. These results indicate that AscendGYM poses a meaningful challenge for further optimizing existing kernels. Per-kernel results are provided in Figure 9.

Budget Metrics. Token consumption, interaction rounds, and wall-clock time exhibit different patterns across combinations, indicating that a single cost measure cannot fully characterize optimization expenditure. Claude Code/Opus 5 consumes fewer tokens than Codex/GPT-5.6 but uses more interaction rounds. OpenCode/Qwen-35B has relatively modest token and round counts, yet takes approximately 11.1 hours per task (Figures 10–12).

Reducing token expenditure and shortening the wait for optimization may therefore favor different choices. Reporting each cost measure separately complements the joint scores and supports decisions under different resource constraints. Detailed statistics are provided in Appendix D.2; cumulative and mean costs are computed from the selected run records and include non-passing runs.

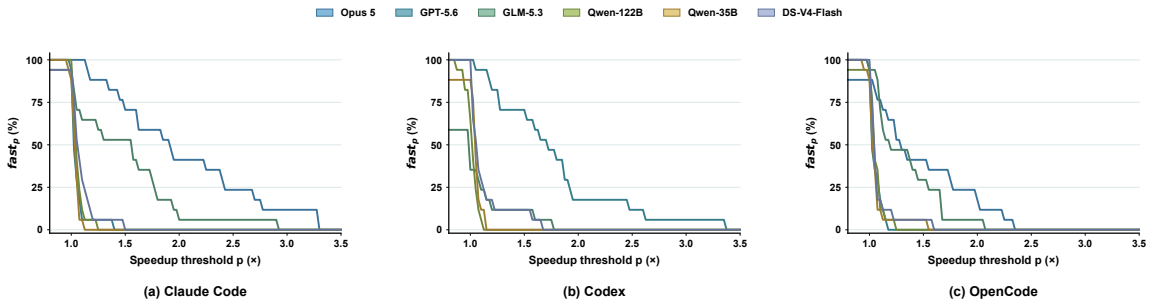


Figure 6: Speedup-threshold attainment (fast$_p$) over full search trajectories.

Reliability Metrics. Task pass rate and performance gains provide different information. GPT-5.6 passes all tasks with each of the three harnesses, yet its GM speedup differs substantially: approximately 1.69× with Codex versus 1.05× with Claude Code (Figures 7 and 5). Producing correct candidates therefore does not guarantee substantial performance gains, and high pass rates do not imply that the optimization challenges have been resolved.

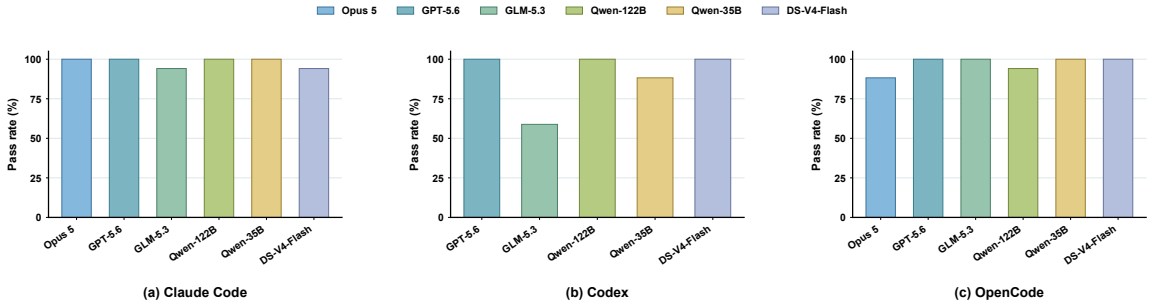


Figure 7: The task pass rate among different combinations.

Some combinations also fail to produce correct candidates on all tasks. Because performance metrics retain the initial implementation when no improvement is found, speedup alone cannot distinguish between a run with no correct candidate and one with correct candidates that do not improve performance. Task pass rate supplies this complementary information, allowing performance results to be interpreted together with the outcomes of the complete evaluation workflow. Task-level accounting is provided in Appendix D.3.

4.2.2 Model Evaluation

We hold the model fixed at GPT-5.6 Luna and compare four `Default` configurations: cold, warm, no lessons, and code only. The task scope and configuration-specific protocols are specified in Section 4.1 and Appendix A.

Joint Performance–Budget Metrics. Warm achieves the highest STS at all three budgets (Figure 8). At 10M tokens, its performance component already exceeds cold's while its cost deduction is smaller, indicating that this configuration obtains greater gains within the limited budget with a lower cost of attaining those improvements.

As the budget increases, cold's performance component grows further and the gap to warm narrows; however, warm retains both a higher score and a larger performance component at 50M tokens (Figure 8). Under their respective evaluation protocols, continued search with cold recovers part of the gap, whereas warm already provides strong optimization results under a tight budget.

Code only outperforms no lessons at all three budgets, but both remain below the full cold configuration (Figure 8). These differences persist with the model held fixed, showing that the `Default` configuration also influences optimization outcomes. The comparison describes each configura-

tion's overall performance under its corresponding evaluation protocol (Appendix A). Comparisons between models using `Default` should therefore keep the settings and prior experience consistent to avoid attributing configuration differences to model capability.

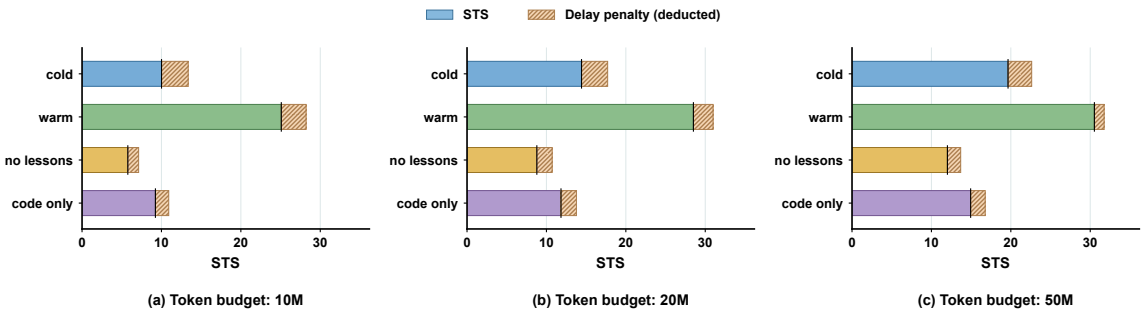


Figure 8: STS under three token budgets using GPT-5.6 Luna. Solid bars show scores; hatched segments show cost deductions.

5 Conclusion

We introduced **AscendGYM**, a kernel optimization benchmark for Ascend NPUs that evaluates agents' ability to further improve existing implementations. Its initial suite comprises 48 kernels from real LLM workloads across three difficulty levels, each defining an optimization task with complete, runnable initial source code and input configurations extracted from the corresponding workloads. Through its built-in execution sandbox, **AscendForge**, AscendGYM provides consistent candidate compilation, correctness checking, and performance measurement, supporting both model comparisons using a fixed Default harness and evaluation of model–harness combinations. We also designed **STS**, which combines final performance gains with cumulative token consumption at each improvement to jointly assess optimization effectiveness and process efficiency.

We systematically evaluated multiple leading models paired with different harnesses on the 17 tasks in Batch 1 of this suite. The results support the benchmark and metric design by showing that the tasks remain challenging for current agents and that jointly assessing performance, cost, and reliability reveals distinctions that speedup alone does not capture. Future work will extend the evaluation to the remaining 31 tasks in Batches 2 and 3 and broaden the task and input coverage to further assess agents' kernel optimization capabilities in real LLM training and inference scenarios.

References

Martin Andrews and Sam Witteveen. Gpu kernel scientist: An llm-driven framework for iterative kernel optimization. *arXiv preprint arXiv:2506.20807*, 2025.

Anthropic. Introducing Claude Opus 5, 2026. URL https://www.anthropic.com/news/claude-opus-5. Accessed: 2026-10-19.

Anthropic. Claude Code: Overview. Official documentation, n.d. URL https://code.claude.com/docs/en/overview. Accessed: 2026-10-19.

Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. Kevin: Multi-turn rl for generating cuda kernels. In *International Conference on Learning Representations*, volume 2026, pp. 83418–83452, 2026.

CANN_ops_cv. ops-cv: CANN computer vision operator library. GitCode repository, n.d. URL https://gitcode.com/cann/ops-cv. Accessed: 2026-10-11.

CANN_ops_math. ops-math: CANN math operator library. GitCode repository, n.d. URL https://gitcode.com/cann/ops-math. Accessed: 2026-10-11.

CANN_ops_nn. ops-nn: CANN neural network operator library. GitCode repository, n.d. URL https://gitcode.com/cann/ops-nn. Accessed: 2026-10-11.

CANN_ops_transformer. ops-transformer: CANN transformer operator library. GitCode repository, n.d. URL https://gitcode.com/cann/ops-transformer. Accessed: 2026-10-11.

Xinzi Cao, Jianyang Zhai, Pengfei Li, Zhiheng Hu, Cen Yan, Bingxu Mu, Guanghuan Fang, Bin She, Jiayu Li, Yihan Su, Dongyang Tao, Xiansong Huang, Fan Xu, Feidiao Yang, Yao Lu, Chang-Dong Wang, Yutong Lu, Weicheng Xue, Bin Zhou, and Yonghong Tian. AscendKernelGen: A systematic study of LLM-based kernel generation for neural processing units, 2026. URL https://arxiv.org/abs/2601.07160.

JooHyoung Cha, Munyoung Lee, Jinse Kwon, Jemin Lee, and Yongin Kwon. Multi-level machine learning-guided autotuning for efficient code generation on a deep learning accelerator. In *Proceedings of the 26th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*, pp. 134–145, 2025.

Yao Cheng, Jianfeng Chen, Jie Chen, Li Chen, Liyu Chen, Wentao Chen, Zhengyu Chen, Shijie Geng, Aoyan Li, Bo Li, et al. FullStack Bench: Evaluating LLMs as full stack coders. *arXiv preprint arXiv:2412.00535*, 2024.

DeepSeek-AI et al. DeepSeek-V4: Towards highly efficient million-token context intelligence, 2026. URL https://arxiv.org/abs/2606.19348.

Xue-Jian Gao, Deng Pan, Yueming Su, Jiasheng Li, Bin Du, Fengming Zhu, Chengdi Ma, Junyi Fan, Qichen Liao, Chengqiu Hu, Xinxian Chen, Lingchao Zheng, Jun Li, Jiwei Yang, and Yuwei Fan. CANN Bench: Benchmarking agent generated kernels against real NPU and algorithmic limits, 2026. URL https://arxiv.org/abs/2607.20518.

Jaber Jaber and Osama Jaber. Autokernel: Autonomous gpu kernel optimization via iterative agent-driven search. *arXiv preprint arXiv:2603.21331*, 2026.

Aditya K Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. Pod-attention: Unlocking full prefill-decode overlap for faster llm inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pp. 897–912, 2025.

Kelun Lei, Hailong Yang, Huaitao Zhang, Xin You, Kaige Zhang, Zhongzhi Luan, Yi Liu, and Depei Qian. PRAGMA: A profiling-reasoned multi-agent framework for automatic kernel optimization, 2025. URL https://arxiv.org/abs/2511.06345.

Dongfang Li, Xiaodong Luo, Ruoyu Sun, et al. SLAI T-Rex: Full-parameter post-training of the DeepSeek-V4 family on Ascend SuperPOD, 2026. URL https://arxiv.org/abs/2607.20145.

Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. TritonBench: Benchmarking large language model capabilities for generating Triton operators. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 23053–23066. Association for Computational Linguistics, 2025a. doi: 10.18653/v1/2025.findings-acl.1183. URL https://aclanthology.org/2025.findings-acl.1183/.

Shangzhan Li, Zefan Wang, Ye He, Yuxuan Li, Qi Shi, Jianling Li, Yonggang Hu, Wanxiang Che, Xu Han, Zhiyuan Liu, et al. Autotriton: Automatic triton programming with reinforcement learning in llms. *arXiv preprint arXiv:2507.05687*, 2025b.

Edward Lin, Sahil Modi, Siva Kumar Sastry Hari, Qijing Huang, Zhifan Ye, Nestor Qin, Fengzhe Zhou, Yuan Zhang, Jingquan Wang, Sana Damani, et al. SOL-ExecBench: Speed-of-light benchmarking for real-world GPU kernels against hardware limits, 2026. URL https://arxiv.org/abs/2603.19173.

OpenAI. GPT-5.6 Luna model. Official model documentation, 2026a. URL https://developers.openai.com/api/docs/models/gpt-5.6-luna. Accessed: 2026-10-19.

OpenAI. GPT-5.6 Terra model. Official model documentation, 2026b. URL https://developers.openai.com/api/docs/models/gpt-5.6-terra. Accessed: 2026-10-19.

OpenAI. Codex CLI. Official documentation, n.d. URL https://developers.openai.com/codex/cli/. Accessed: 2026-10-19.

OpenCode Contributors. OpenCode: Intro. Official documentation, n.d. URL https://opencode.ai/docs/. Accessed: 2026-10-19.

Anne Ouyang, Simon Guo, Simran Arora, Alex L Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 47356–47415. PMLR, 2025. URL https://proceedings.mlr.press/v267/ouyang25a.html.

Qwen Team. Qwen3.5-122B-A10B. Hugging Face model card, 2026a. URL https://huggingface.co/Qwen/Qwen3.5-122B-A10B. Accessed: 2026-10-19.

Qwen Team. Qwen3.6-35B-A3B. Hugging Face model card, 2026b. URL https://huggingface.co/Qwen/Qwen3.6-35B-A3B. Accessed: 2026-10-19.

Arya Tschand, Muhammad Awad, Ryan Swann, Kesavan Ramakrishnan, Jeffrey Ma, Keith Lowery, Ganesh Dasika, and Vijay Janapa Reddi. Swizzleperf: Hardware-aware llms for gpu kernel performance optimization. *arXiv preprint arXiv:2508.20258*, 2025.

vLLM Ascend Contributors. vLLM Ascend: A hardware plugin for vLLM on Huawei Ascend. GitHub repository, n.d. URL https://github.com/vllm-project/vllm-ascend. Accessed: 2026-10-11.

Jianghui Wang, Saptarshi Majumder, Umang Pandey, Chao Xu, Bin Ding, Manoj Rao, Ziqiong Liu, Pratik Prabhanjan Brahma, Dong Li, Zicheng Liu, and Emad Barsoum. GEAK-Triton v2 family of AI agents: Kernel optimization for AMD Instinct GPUs. AMD ROCm Blog, December 2025. URL https://rocm.blogs.amd.com/artificial-intelligence/geak-agents-family/README.html. Accessed: 2026-10-20.

Zhongzhen Wen, Yinghui Zhang, Zhong Li, Zhongxin Liu, Linna Xie, and Tian Zhang. MultiKernelBench: A multi-platform benchmark for kernel generation, 2025. URL https://arxiv.org/abs/2507.17773.

Shanli Xing, Yiyan Zhai, Alexander Jiang, Yixin Dong, Yong Wu, Zihao Ye, Charlie Ruan, Yingyi Huang, Yineng Zhang, Liangsheng Yin, Aksara Bayyapu, Luis Ceze, and Tianqi Chen. FlashInfer-Bench: Building the virtuous cycle for AI-driven LLM systems, 2026. URL https://arxiv.org/abs/2601.00227.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Lingyun Yang, Yuxiao Wang, Shenghao Liang, Linfeng Yang, Daocheng Ying, Chunbo You, Rui Zhang, Luping Wang, Yinghao Yu, Guodong Yang, and Liping Zhang. Are LLM-generated GPU kernels production-ready? a trace-driven benchmark and optimization agent, 2026. URL https://arxiv.org/abs/2607.14541.

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. Flashinfer: Efficient and customizable attention engine for llm inference serving. *Proceedings of Machine Learning and Systems*, 7, 2025.

Z.ai. GLM-5.3: Frontier coding with emergent cyber capabilities, 2026. URL https://z.ai/blog/glm-5.3. Accessed: 2026-10-19.

A Default Harness Configuration

`Default` is based on AuraKernel and uses GPT-5.6 Luna with reasoning effort set to `xhigh`. The evaluation protocol covers the 17-task suite described in Section 3.1 and includes four configurations: cold, warm, no lessons, and code only.

Configurations and Experience Management. Table 1 summarizes experience initialization, online experience accumulation, and search policies. Agent-search disables dynamic search-graph exploration and joint proposals from multiple parent implementations. Context is reconstructed for each implementation cycle, while search history and enabled experience stores persist across cycles.

Table 1: Default configurations.

Variant	Initial experience	Online accumulation	Search policy
Cold	None	Strategy + coding	Full Default
Warm	5 strategy lessons	Strategy + coding	Full Default
No lessons	None	None	Agent-search
Code only	None	Code only	Agent-search

For CANN tasks, warm imports five strategy lessons from other kernels. For `fused_gdn_gating`, `hc_post`, and `hc_pre_sinkhorn`, warm imports five validated strategy lessons with positive measured gains from a cold run of the same kernel. Imported lessons are fixed before search. When reusing experience for the same kernel, warm starts from the task's initial implementation without inheriting cold's optimized code or search checkpoint; experience preparation costs and warm search costs are recorded separately.

Execution and Measurement Settings. Each implementation cycle allows at most ten profiling calls, with a stagnation window of five and a relative-improvement threshold of 0.005. The harness does not override temperature or top-p and uses the model service's effective settings.

For CANN tasks, each measurement uses eleven executions in warm and eight in the other configurations. All configurations for the three vLLM Ascend tasks use eight executions. The first three executions serve as warm-up, and the remaining executions are averaged to obtain latency. Configuration comparisons reflect their respective budgets, experience sources, and measurement settings.

B Evaluation Prompt

Model–harness evaluation supplies a shared task prompt to the native coding agent, while retaining each harness's own system prompt and tool interfaces. The following abridged English translation of the original Chinese instructions presents the task context, optimization workflow, and evaluation constraints. Task-specific values and deployment paths are represented by placeholders; operator specifications, source files, and tests are provided separately.

```
You are a native coding agent participating in a kernel optimization experiment.
    Use your own file-reading, search, editing, and terminal tools. Do not output
    custom single-step tool JSON or wait for an external program to direct each
    step.

Task: <kernel_name>
Working directory: <workspace>
Operator specification: <spec_path>
Input manifest: <manifest_path>
Editable files: <editable_paths>
Test scripts: <test_paths>

Only the listed kernel source files may be modified. Other manifests, tests,
    specifications, golden references, and evaluation programs must remain
    unchanged. Only files in the input manifest enter the build; temporary files
    are not submitted as candidates.
```

```
Use your native terminal tool to run:
  baseline: <baseline_command>
  candidate: <candidate_command>
  status: <status_command>
These commands do not invoke a language model.

Optimization workflow:
1. Read the specification, implementation, and tests to understand the semantics
    and constraints. Run the baseline before editing the kernel implementation.
2. After the baseline passes, use actual evaluation reports to propose
    optimizations and edit the kernel source files in the workspace.
3. Finish editing before submitting a candidate. Do not modify source files during
    evaluation; the evaluator freezes a submission snapshot.
4. For compilation or correctness failures, inspect the diagnostics, fix the
    problem, and resubmit. Failed evaluations also consume the candidate budget.
5. Every passing version is backed up, even if it is slower. While budget remains
    and no stop is required, you may continue exploring or revert to an earlier
    version.
6. When the remaining budget is zero or stop_required=true, stop evaluating,
    editing, and reverting code. Summarize and finish; the controller delivers the
    best validated saved snapshot.
7. An interrupted HTTP connection does not imply that evaluation did not run.
    Retain the request_id and query or retry the original call using the same ID.
8. Do not change tolerances, golden references, test coverage, operator
    interfaces, shapes, or data types. Never present unvalidated code as the best
    validated implementation.

Search instructions:
- The candidate evaluation limit is 80; baseline measurements do not consume it.
- Within this limit, aim to implement and evaluate 50 substantively different
    optimization proposals.
- A proposal is a code change submitted for evaluation, not a written suggestion.
    Proposals may build on the best validated version; they need not be
    independent, correct, or faster.
- Compilation and correctness repairs to the same proposal do not count as new
    proposals, although each evaluation consumes budget. Duplicate code and
    cosmetic changes do not count as new proposals.
- Continue while budget and reasonable optimization directions remain. Reaching
    the soft target does not require stopping; hard stopping rules take
    precedence. Explain the reason for ending early.

Measurement and reporting:
- Follow the fixed profiling protocol: eight executions, including three warm-up
    executions; use the remaining executions for the arithmetic mean. All
    executions must pass correctness checks.
- There is no hard token or total wall-clock limit. Token consumption and elapsed
    time are recorded externally; do not estimate or fabricate them.
- Before each modification, briefly state the optimization hypothesis and expected
    profiling effects. Use actual diagnostics to explain subsequent fixes.
- Preserve evaluation outputs and request IDs. Do not modify the evaluator,
    traces, snapshots, backups, configuration, or .bench directory, or bypass the
    evaluation entry point.
- End with a brief report of the proposal count, best validated evaluation_id,
    changes, and results. If the soft target was not reached, explain why.
- Final re-evaluation is performed by the controller, outside the search budget,
    and is not returned as feedback for further optimization.
```

C Hardware and Software Environment

The evaluation environment uses Ascend 910C NPUs and the software stack listed in Table 2. Each evaluation request is assigned one NPU, which is released when evaluation finishes.

Table 2: Hardware and software environment for AscendForge kernel evaluation.

Component	Configuration or version
Hardware and host system	
NPU	Ascend 910C
Runtime device identifier	`Ascend910_9392`
Host architecture	AArch64
Linux kernel	5.10.0
Runtime and build tools	
Python	3.11.15
PyTorch	`2.10.0+cpu`
`torch_npu` backend	2.10.0.post2
CANN	9.0.1
GNU C/C++	11.4.0
Ascend C compiler	`ccec`, Clang 15.0.5
Profiler	`msprof` 26.0.0
Device stack (21 October 2026)	
NPU driver	25.5.5
NPU firmware	7.8.0.9.220

D Supplementary Model–Harness Results

D.1 Per-Kernel Speedup

Figure 9 provides the task-level breakdown of the GM speedup and fast_p results in Section 4.2.1, using the same evaluation records. Each bar reports the best correctness-validated speedup retained in the corresponding run, with the initial implementation retained when no improvement is found.

The gains vary substantially across tasks. The best observed speedup reaches approximately 3.29× on `fused_gdn_gating`, compared with 1.17× on `moe_gating_top_k`. Moreover, the median speedup across supported combinations remains below 1.1× on 13 tasks. Substantial gains on selected kernels therefore coexist with limited improvements across much of the evaluated task–configuration space. These results highlight the task-dependent optimization challenges posed by AscendGYM and show how per-kernel results complement aggregate scores by revealing the distribution of gains.

D.2 Resource Costs

We compare token consumption, model interaction rounds, and wall-clock time using the same records as the model–harness evaluation. Each supported model–harness combination retains the latest record for each task. We report cumulative costs and per-task means over these records, including non-passing runs.

Token Consumption. As shown in Figure 10, Codex/GPT-5.6 consumes approximately 140.4M tokens per task, compared with 81.75M for Claude Code/Opus 5, yet the latter achieves higher GM speedup. Performance rankings therefore do not simply follow token consumption, and the two should be considered together when assessing the effectiveness of optimization effort.

Model Interaction Rounds. Claude Code/Opus 5 uses more model interaction rounds per task than Codex/GPT-5.6 while consuming fewer tokens (Figures 10 and 11). Dividing total tokens by total recorded rounds gives an approximately threefold higher token cost per round for the latter. The two metrics thus characterize complementary aspects of resource use: interaction frequency and token expenditure. Reporting both helps distinguish longer sequences of lighter interactions from shorter sequences with greater token use per round. Here, rounds refer to recorded model interactions, rather than the number of candidate implementations or kernel evaluations.

Wall-Clock Time. Elapsed time also distinguishes configurations with similar gains and token consumption. Qwen-35B achieves GM speedups of approximately 1.06× with OpenCode and 1.05× with Codex, with comparable mean token consumption, yet takes 11.1 and 2.2 hours per

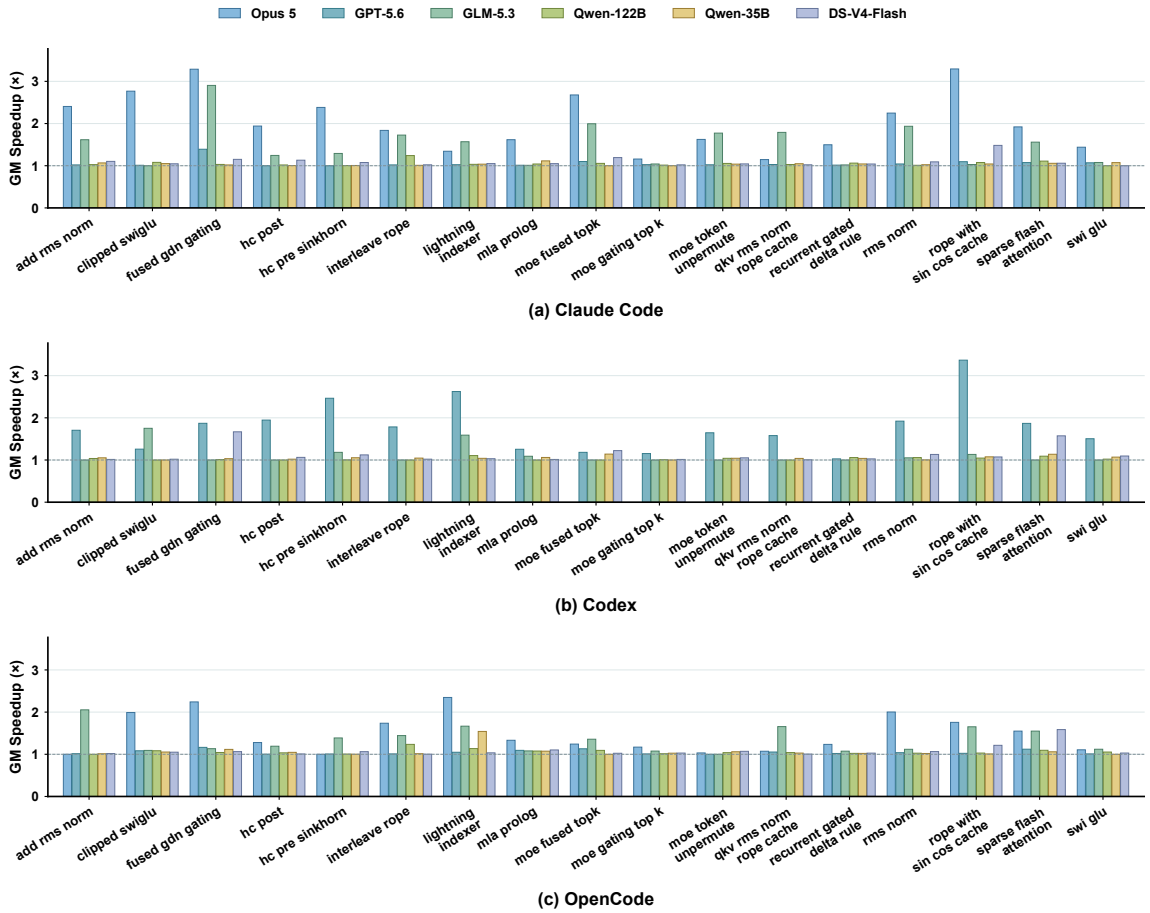


Figure 9: Per-kernel speedup. Panels show Claude Code, Codex, and OpenCode; dashed lines mark $1\times$.

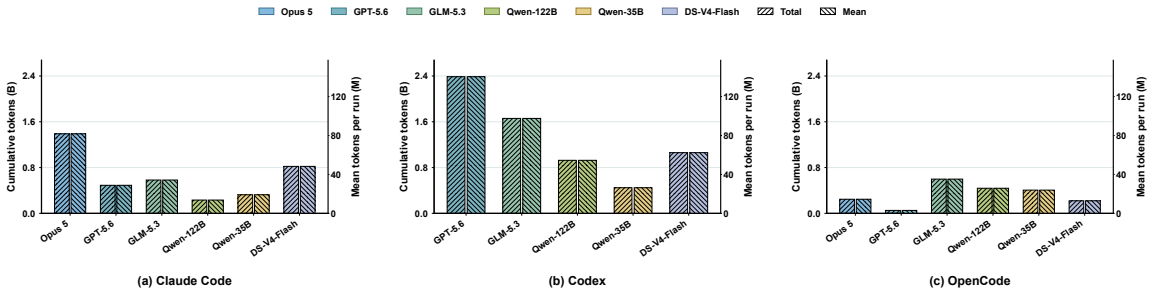


Figure 10: Token consumption. Bars show totals in billions (left axis) and per-task means in millions (right axis).

task, respectively (Figures 5, 10, and 12). This roughly fivefold difference shows that similar token expenditure and final performance can entail substantially different turnaround times. Wall-clock measurements consequently support configuration choices when optimization must finish within an operational time window.[1]

[1]The Codex/GLM-5.3 record for `clipped_swiglu` includes an estimated backend completion tail.

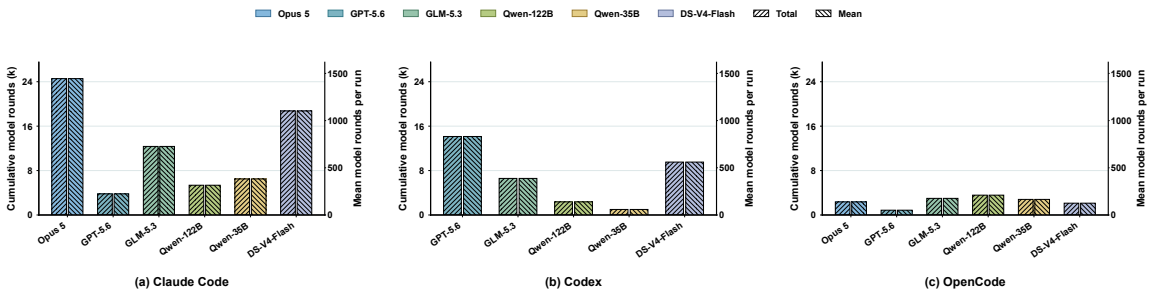


Figure 11: Model interaction rounds. Bars show totals in thousands (left axis) and per-task means (right axis).

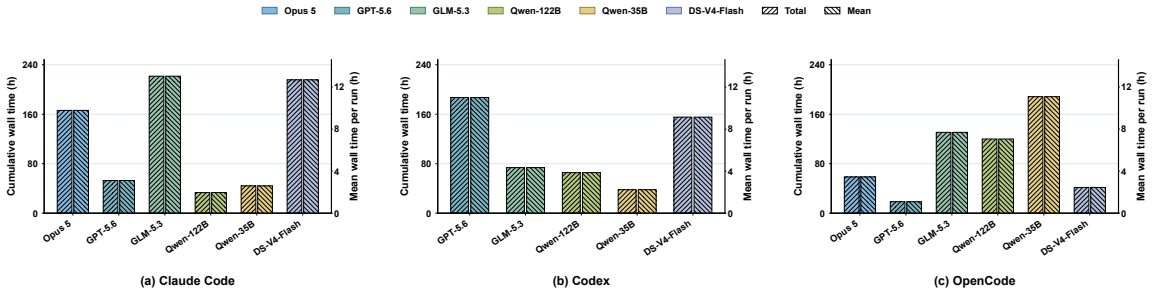


Figure 12: Wall-clock time. Bars show summed run hours (left axis) and mean hours per task (right axis).

D.3 Task Pass-Rate Accounting

The task pass rates in Figure 7 follow the definition in Section 3.3. Across all selected records, 31 retain the baseline speedup of $1\times$: 17 produced correct candidates without a performance improvement, while 14 produced no correctness-validated candidate. These records share the same retained speedup but differ in correctness outcomes. Reporting pass rates alongside speedup captures this distinction and provides a fuller account of optimization behavior.

Codex/GLM-5.3 illustrates this accounting in more detail. It passes 10 of 17 tasks. Among the seven non-passing tasks, five have no candidate submissions, while `moe_token_unpermute` and `recurrent_gated_delta_rule` have submissions but no correctness-validated candidates. All tasks remain in the denominator. This breakdown records outcomes at the candidate-submission and correctness-validation stages, complementing the performance assessment of the complete model–harness workflow.